

Bugs y Testing

Un bug no es un accidente. Es una pregunta que no te hiciste antes.

1 · Idea central de la sesión

Un bug no es un error del ordenador. Es un caso que el sistema no sabe manejar porque nadie lo pensó antes. El testing no consiste en pulsar botones hasta que algo falle: es **razonar sobre qué puede salir mal antes de que salga mal**.

Al terminar esta sesión debes llevarte una idea clara:

"Antes de ejecutar código, piensa en los casos que el sistema no espera."

2 · Conexión con lo que ya sabes

En Scratch...	En nuestra app...
Si un sprite se movía fuera del escenario, el juego fallaba en silencio.	Si el usuario pulsa una casilla ya ocupada en Tic Tac Toe, ¿qué pasa?
No había forma de ver qué estado tenía el programa en ese momento.	El estado centralizado hace que los bugs sean visibles y trazables.
Los bloques 'si / si no' cubrían casos felices, rara vez los raros.	Los casos borde son exactamente los que el patrón evento → estado →

3 · El concepto: qué es un bug y cómo se piensa

Un **bug** aparece cuando el sistema recibe una entrada o se encuentra en un estado que nadie diseñó para manejar. No es que el ordenador se equivoque: es que **el diseño tenía un hueco**.

Caso no contemplado	Una acción que el código no espera.	Pulsar 'Jugar' dos veces seguidas.
Estado incoherente	El estado dice una cosa, la UI muestra otra.	pantallaActual = 'menu' pero se ve el tablero.
Efecto colateral	Una función modifica algo que no debería.	navegar() reinicia la puntuación sin querer.

4 · Diseñar el testing antes de ejecutar

El testing manual consiste en **construir una lista de preguntas** sobre el sistema antes de tocarlo. Para cada función importante, pregúntate:

- ¿Qué pasa si el usuario hace esto en el momento equivocado?
- ¿Qué pasa si el estado tiene un valor que no esperamos?

- ¿Qué pasa si esta función se llama dos veces seguidas?
- ¿Qué pasa si el usuario pulsa muy rápido?
- ¿Qué pasa si nunca llega a pasar por este estado?

Patrón fundamental — dónde buscar los bugs:

```
evento → función → estado cambia → UI se actualiza
↑ ↑ ↑
¿qué evento ¿qué cambia ¿qué muestra
no esperamos? mal aquí? la UI si falla?
```

5 · Aplicación al proyecto — lista de tests manuales

Para la app de mini-juegos, los tests manuales se organizan por módulo. Cada test es una pregunta con un resultado esperado:

Módulo	Test ("¿qué pasa si...?")	Resultado esperado
Navegación	Pulso 'Jugar' desde el menú	pantallaActual = 'juego-tic-tac-toe' UI muestra el tablero
Navegación	Pulso 'Volver' a mitad de partida	Vuelvo al menú. El tablero se resetea.
Tic Tac Toe	Pulso una casilla ya ocupada	No ocurre nada. Turno no cambia.
Tic Tac Toe	X gana en diagonal	Se detecta victoria. Juego se bloquea.
Tic Tac Toe	Tablero lleno sin ganador	Se detecta empate. Mensaje visible.
Ahorcado	Introduzco la misma letra dos veces	No cuenta como error extra.
Ahorcado	Introduzco un número o símbolo	Se ignora la entrada. Mensaje de error.
Estado	Inicio un juego sin pasar por el menú	El estado es coherente. No hay datos basura.

6 · Uso guiado de IA

✓ Prompts que tienen sentido	✗ Prompts que hay que evitar
"¿Qué casos borde no estoy cubriendo en la función validarMovimiento() de Tic Tac Toe?"	"Arregla el bug." (sin describir el bug ni el contexto)
"Tengo este test fallido: [descripción]. ¿Por qué puede estar pasando?"	"Hazme una lista de todos los tests que necesita la app."
"¿Esta función puede modificar el estado sin querer? Revísala."	"Soluciona todos los bugs de mi código."

■ Qué revisar siempre cuando la IA propone un fix:

- ¿La solución entiende el bug, o solo lo oculta?
- ¿Modifica el estado de forma que rompe otro módulo?
- ¿El fix crea un caso borde nuevo que no habías previsto?

Regla: La IA puede encontrar el bug contigo. No puede pensar por ti en qué casos pueden fallar.

7 · Preguntas de pensamiento crítico

1. ¿Por qué es importante separar la lógica de la UI para poder hacer tests?

Pista: Si la lógica y la UI están mezcladas, no puedes probar una sin activar la otra.

2. ¿Qué pasa si encuentras un bug que no está en tu lista de tests?

Pista: Significa que tu lista tiene huecos. Añade ese caso. El sistema de tests crece con los bugs reales.

3. ¿Puede haber código sin bugs si haces suficientes tests?

Pista: No. Los tests reducen los bugs conocidos, no garantizan que no existan bugs desconocidos.

4. ¿Por qué navegar() no debería reiniciar el estado del juego?

Pista: Porque su única responsabilidad es cambiar pantallaActual. Mezclar responsabilidades crea bugs de efecto colateral.

8 · Mini-reto de la sesión (1 hora)

Objetivo: construir la lista de tests manuales de tu app.

Parte 1 — Diseño (20 min):

Sin tocar código, escribe en papel o en un doc una lista de al menos 10 preguntas del tipo «¿qué pasa si...?» para los módulos de navegación, Tic Tac Toe y Ahorcado. Para cada pregunta, escribe también el resultado que esperas.

Parte 2 — Ejecución (25 min):

Abre la app y ejecuta cada test. Marca cada uno como ✓ (funciona), ✗ (falla) o ? (comportamiento raro). Si fallas, describe exactamente qué pasó.

Parte 3 — Análisis (15 min):

Para cada ✗ o ?, identifica a qué parte del patrón evento → función → estado → UI pertenece el fallo. ¿Es un caso no contemplado, un estado incoherente o un efecto colateral?

Resultado esperado: una tabla de tests con estado real de la app — no código nuevo, sino criterio.

9 · Qué NO hacer — antipatrones

✓ Correcto	✗ Incorrecto / Antipatrón
Describir exactamente qué hace el bug y en qué estado ocurre.	"No sé qué falla, pero algo falla." → La IA no puede ayudarte así.
Probar los casos borde primero (lleno, vacío, repetido).	Probar solo el camino feliz (el flujo normal sin errores).
Documentar el test aunque todo funcione bien.	Borrar la lista de tests cuando todo parece funcionar.
Parar cuando el bug no está claro y analizar el estado antes de arreglarlo.	Añadir código hasta que deje de fallar sin entender por qué fallaba.

La idea que te llevas hoy

Testear no es romper cosas. Es hacerse las preguntas correctas antes de que el usuario las haga.